

Finetuning a Domain-Specific Chatbot: Trench Crusade using Mistral AI

Jacob Helzner

Shiva Ahir

Viswadruth Akkaraju

ESE 577: Deep Learning Algorithms & Software

Instructor: Prof. Jorge Mendez-Mendez

Abstract

This project introduces a domain-adapted, AI-driven narrative engine for the tabletop universe Trench Crusade, inspired by AI Dungeon and powered by a finetuned Mistral-7B model. Because Trench Crusade (est. 2023) is absent from the model's pretraining corpus, standard LLMs hallucinate lore or conflate it with adjacent properties like Warhammer 40k. To address this, we construct a supervised dataset from official sources and augment it with Gemini-generated question-answer pairs, performing human evaluation to filter hallucinations and evaluate model accuracy. We then execute a two-stage LoRA finetuning pipeline: initial lore-alignment stage followed by gameplay tuning on 77 game transcripts. We evaluate the model using both quantitative metrics and human playtesting (n=10). Our results demonstrate that while the baseline model suffers from severe "historical drift" (e.g., referencing real-world WWII history), the finetuned model achieves approximately 65% accuracy on domain-specific fact retrieval vs 5% on untuned mistral.

1

1 Introduction

Large language models (LLMs) such as Mistral-7B provide strong general-purpose language capabilities but are not automatically experts on niche domains that appear after their pretraining cutoff. Trench Crusade is a tabletop game whose detailed lore, factions, and mechanics are primarily described on its official website and related recent materials. The likelihood that these texts are fully represented in the Mistral pretraining corpus is low, especially because the model was trained on data up to 2023, while the game universe continues to evolve.

Our assignment is to specialize Mistral-7B to a domain of our choice using parameter-efficient finetuning (PEFT), LoRA, and quantization to keep GPU memory requirements within the limits of Google Colab. The goal is to:

- choose a niche domain,
- build a suitable dataset (either text-completion or Q&A),
- finetune Mistral-7B using LoRA and quantization, and
- evaluate the resulting model numerically and through human gamers and taking their survey.

In this project, our domain is the *Trench Crusade* game world. Our specialization target is a chatbot that can:

- answer lore questions (characters, factions, setting, events),
- clarify game rules that are described in the official materials, and

¹ Author order chosen by dice role, no correlation with work contribution

- support a planned pipeline of game JSON scenarios that players can interact with and rate.

We consider two complementary training viewpoints:

1. **Chatbot / instruction tuning:** we format data as Q&A pairs for `Mistral-7B-Instruct`.
2. **Next-token prediction:** we treat the lore as plain text and train the model to predict the next token in each sequence, which is closer to the original pretraining objective.

We also explicitly analyze what it means to *finetune an already finetuned model* (`Mistral-7B-Instruct`) on top of our new Trench Crusade data, and how that differs from finetuning the base `Mistral-7B` model.

2 Domain Analysis and Data Collection

2.1 Trench Crusade and The "Hallucination" Problem in Niche IPs

Trench Crusade is a grimdark alternate-history game universe with specific factions, lore events, and stylistic language. This structure makes it an ideal domain for assessing the limitations of pre-trained LLMs: it is text-heavy and highly structured but too niche and recent (est. 2023) to be well-represented in their training corpora.

Our assumption is that:

- The base Mistral model has seen very little of this specific lore (because of recency and niche nature).
- Therefore, finetuning should give a noticeable improvement in answering game-specific questions.

During a control test, we observed that even high-capacity frontier models (e.g., Gemini, GPT-4) frequently hallucinated domain-incorrect details or merged Trench Crusade lore with unrelated grimdark or historical properties. This behavior manifested in several ways:

- Blending the setting with Warhammer 40k (e.g., introducing factions reminiscent of the Imperial Guard or Mechanicus)
- Mixing in elements from the Scythe and Dune universe, For example, the model repeatedly referenced an event it called the 'Clockwork Heresy', which on closer inspection was a retelling of the *Butlerian Jihad* from the Dune series.
- More surprisingly, Gemini introduce real-world historical details. For example, when asked to generate an adventure scenario "strictly set in the Trench Crusade universe," Gemini produced a storyline involving an Indian Sepoy, a character class that does not appear anywhere in Trench Crusade lore. The Trench Crusade universe makes no mention of India, yet contains geographically adjacent analogues (e.g., the Iron Sultanate, the fractured remnants of Britannia), which likely contributed to semantic drift toward real historical references.

To systematically measure this phenomenon, we issued the prompt: *"Generate ten scenarios for an AI-dungeon style text-based adventure game strictly set in the Trench Crusade universe."* We repeated this prompt after every ten generated games, for a total of ten iterations (100 scenarios). Across iterations, we observed two consistent patterns:

- **Looping Behavior:** after approximately 60–70 scenarios, Gemini began producing nearly identical scenario templates, suggesting it was falling back to generic narrative priors rather than domain-specific lore
- **Increasing Implausibility:** beyond this point, the model introduced progressively more implausible or non-canonical elements, often drifting into real-world historical events or entirely unrelated fictional tropes.

To contextualize these observations, we repeated the exact experiment using a more established and extensively documented fictional property, *Warhammer 40,000*, whose lore spans multiple decades and is well represented in public text corpora. Under identical prompting conditions, Gemini exhibited *neither* looping behavior across at least 1,000 generated scenarios *nor* any cross-property conflation, in contrast to its performance on Trench Crusade. This comparison suggests that hallucination severity correlates strongly with the scarcity of an IP’s presence in LLM pretraining data.

As a caveat, we acknowledge that Warhammer 40,000 might eventually produce similar degradation or looping patterns if pushed far enough. However, identifying a precise upper bound would have required generating tens of thousands of scenarios beyond the scope and time constraints of this class project. We therefore selected 1,000 scenarios as a practical cutoff: an order of magnitude larger than our Trench Crusade experiment and sufficiently large to support, with reasonable confidence, the conclusion that the hallucination effects we observed can be attributed to limited pretraining exposure rather than inherent model behavior.

Figure 1 (pictured below) shows an example output batch, where red boxes highlight implausible or hallucinated scenarios. These qualitative failures further reinforce the conclusion that Trench Crusade is underrepresented in pretraining corpora, and that domain-specific finetuning is necessary to align LLM outputs with canonical lore.

2.2 Lore Extraction and Paragraph Segmentation

We first collected the raw text:

- Crawled the official Trench Crusade website.
- Extracted all text from lore pages: world background, factions, key characters, and setting descriptions.
- Cleaned the HTML and removed navigation, headers, and unrelated boilerplate.

We then converted the cleaned content into paragraph-level `.txt` files:

- Each paragraph corresponds to a coherent block of lore (e.g., a faction description, a location, a specific event).
- Very short bullet points or captions were merged so that each paragraph contains enough context for a meaningful question.

These `.txt` files form the base corpus from which we derive both QA data and, optionally, text-completion sequences. In total, we collected approximately 60 paragraph-level `.txt` files, each containing on average 300 words of canonical lore about the universe, its factions, and key entities. The size of the corpus is inherently constrained by the recency of the Trench Crusade IP and the limited amount of officially released material within its first two years of existence. As a result, the available lore is comparatively sparse, further motivating the need for synthetic data generation and careful domain-specific finetuning.

2.3 Q&A Generation Using Gemini

Instead of manually writing hundreds of Q&A pairs, we used Google Gemini to generate synthetic questions and answers from the paragraphs. The pipeline is:

1. Combine all paragraph `.txt` files into a single list.
2. For each paragraph, send a prompt to Gemini that asks it to:
 - read the paragraph,
 - generate *one* insightful question whose answer requires understanding that paragraph, and
 - produce an answer that can be supported purely by the given text.
3. Ask Gemini to output the Q&A in JSON format, e.g.:

Character Focus (Faction)	Objective/Mission	Heretic Opposition	Desired Outcome
Prussian (Infantry)	Rescue a trapped Faithful unit.	Trench Zombies (Horde)	Pyrrhic Victory
Papal (Acolyte/ Medic)	Secure and purify a relic.	Tunnel Creeper (Ambush)	Failure
Iberian (Exorcist)	Psychic interrogation of a prisoner.	Warlock (Counter-Psionics)	Success
Norse (Beserker)	Hold a forward listening post.	Crude Cultists (Melee Swarm)	Pyrrhic Victory
Colonial (Ranger)	Patrol and sabotage enemy wire.	Heretic Snipers (Pinning Fire)	Failure
Imperial Russian (Sapper)	Defuse a complex Heretic explosive.	Black Grail Agent (Sabotage)	Success
Celtic (Druid)	Clear a corrupted sector of the trench.	Trench Guardians (Elite Defense)	Pyrrhic Victory
Indian (Sepoy)	Escort a supply convoy through a blizzard.	Trench Zombies (Blizzard Ambush)	Success
Venetian (Inventor)	Retrieve a fallen prototype weapon.	Infernal Sentinels (Patrol)	Failure
Prussian (Receptor)	Silent capture of a Warlock's phylactery.	Warlock/Cultists (Ritual Defense)	Success

Figure 1: Hallucinations in Gemini Scenarios

```
{
  "question": "...",
  "answer":  "...",
}
```

4. Save all Q&A objects into a JSONL file to be consumed by the Mistral finetuning code.

We used the Jupyter notebook provided by Prof. Mendez-Mendez as the base, slightly modifying it to:

- read all our paragraph `.txt` files,
- call Gemini for each paragraph, and
- format the results in the JSON structure expected by the Mistral training framework.

This process yielded a dataset of Trench Crusade specific QA pairs suitable for instruction-style finetuning. However, we note that Gemini itself frequently hallucinated answers even in cases where the input paragraph was unambiguous. Some of these errors were surprisingly dramatic: for example, several responses referred to the Axis powers from World War II as if they were factions within the Trench Crusade universe. In other cases, Gemini blended themes from adjacent grimdark properties. One representative example involved a question about an enemy faction known as the “Plague Daemons.” Gemini initially produced a lore-consistent answer, but midway through began introducing terminology drawn from the *Warhammer 40,000* setting specifically referencing “Plague Legions,” a concept absent from Trench Crusade but present in the Warhammer universe. The resulting answers were often internally coherent but canonically incorrect, requiring manual verification against the official lore.

These problems occurred frequently enough that we thought of manually reviewing all Gemini-generated [X] QA pairs to identify and remove examples contaminated by hallucinations or cross-property. Due to the sheer volume of generated questions, we weren’t able to do this and acknowledge deficiencies in the training data. A similar pattern of hallucination emerged when evaluating outputs from our finetuned Mistral model: although improved, it still occasionally merged properties sometimes subtly, sometimes in ways that were nonsensical. To quantify this, we manually evaluated a subset of [N] Mistral-generated QA pairs after each finetuning round.

This manual review process became one of the largest time sinks in the project, but it was essential for establishing reliable accuracy metrics. After extensive hyperparameter tuning and iterative filtering, our final Mistral model achieved approximately 77 percent accuracy on lore grounded QA evaluation.

2.4 Game JSON Data for Interactive Evaluation

In addition to lore Q&A, our long-term goal is to fine-tune the model on actual game JSON files:

- We collect JSON files describing small games or scenarios (custom-created ones).
- Each JSON encodes a self-contained mini-game or interactive scenario, including story beats, choices, and outcomes.
- These JSONs will later be used to:
 1. fine-tune the model in batches of 10 games at a time, and
 2. run human play-tests where players interact with the model and rate the quality.

In the current phase, we primarily focus on the lore-based Q&A training, with the JSON-based game finetuning defined as the next stage of the project.

2.5 Sample Gemini Prompt for Game Transcript Generation

For the game-style data, we also design a structured system prompt for Gemini to generate short Trench-Crusade-themed interactive transcripts that can later be converted into JSONL training examples. A simplified version of the prompt we use is shown below:

You are an AI specialized in generating interactive, text-based adventure game transcripts based on the Trench Crusade setting. Your goal is to create high-quality training data for a conversational model.

Core Constraints and Requirements Lore and Theme: Adhere strictly to the established lore of the Trench Crusade (Black Guard, Gothic Cardinalate, Iron Sultanate, Court of the Seven-Headed Serpent, Clockwork Heresy, Cannibal Cults, Void Engines, etc.). The tone must be intensely grimdark, horrific, and shocking (e.g., cannibalism, self-sacrifice, religious/machine corruption, extreme violence).

Game Structure: Each game must be a self-contained narrative lasting 10 turns or less. Each turn begins with a Narrator (AI) response describing the consequences of the user's last action, followed by a Your Turn (Player Action) prompt offering 2-3 distinct choices.

Mechanical Realism (Non-Determinism): Do not make outcomes deterministic. Inject elements of chance, surprise, and failure by implementing a hidden dice-roll mechanic. When a roll determines success, failure, or chaos, narrate the result but log the roll in JSON. Include unwinnable or catastrophic scenarios.

Character and Scenario Variety: Rotate perspectives (Black Guard Captain, Iron Sultanate Engineer, Heretic Apostate, Gothic Cardinalate Commissar, etc.).

Final Output: Conclude with an In-Lore Mission Report summarizing character, outcome, and mission status (Success/Failed/KIA/Ascended).

Strict JSON Lines Output Formatting: After the game concludes, output the entire transcript as:

```
"text": " ... "
```

Use '<s>' and '</s>' for turn boundaries. Use '[INST]' and '[/INST]' for user prompts. Use '[SYSTEM: SCENARIO START: ... END SCENARIO]' for the scenario header. Use '[SYSTEM: [ROLL LOG: ...]]' immediately after a choice to log dice outcomes.

Wait for the user to answer | this is a user-generated game.

We used our lore paragraphs as training sequences. Because an individual paragraph can contain many words, a single training example provides a large number of token-level predictions. This is what we mean when we say that “an example has a lot of words in it” for next-token prediction: we slide through the entire paragraph and compute a loss at every token position, instead of only on the answer segment as in the Q&A objective.

Comparison with Q&A.

- **Q&A objective:** explicitly teaches the model to produce short, direct answers when given a question. It is better aligned with chatbot behavior.
- **Next-token objective:** reinforces the model's understanding of the style and content of the lore in a more unsupervised way, but does not directly shape question answering.

In our project, Q&A finetuning is the primary objective. Next-token prediction was explored as an alternative (and potentially complementary) way to inject lore knowledge into the weights.

3 Fine-Tuning Setup

3.1 Base Models and “Fine-Tuning on an Already Fine-Tuned Model”

Mistral-7B comes in multiple variants:

- `Mistral-7B-v0.1` (base model): trained only on next-token prediction, no chat alignment.
- `Mistral-7B-Instruct-v0.1/v0.2`: further finetuned on instruction-following data to act as a chatbot.

There are two conceptually different choices:

Option 1: Start from the base model. We could take `Mistral-7B-v0.1` and finetune it on our Trench Crusade Q&A pairs. In that case, our project is:

- The *first* instruction finetuning layer on top of the base next-token model.
- Responsible both for turning the model into a chatbot and for injecting Trench Crusade knowledge.

The downside is that our dataset is small compared to the original instruction-tuning corpus, so we risk ending up with a model that is good at our niche domain but not very robust as a general chatbot.

Option 2: Fine-tune an already fine-tuned Instruct model. In practice, we focus on `Mistral-7B-Instruct` and *add* a second finetuning layer on top of it (via LoRA). This is what “fine-tuning on an already fine-tuned model” means:

- The base weights already encode general instruction-following behavior and broad world knowledge.
- Our LoRA adaptation only nudges the model toward Trench Crusade-specific patterns.
- We keep the original weights frozen and train only the low-rank adaptation parameters.

Concretely, if W_{instruct} are the weights of `Mistral-7B-Instruct`, LoRA defines:

$$W_{\text{finetuned}} = W_{\text{instruct}} + W_{\text{left}}W_{\text{right}},$$

with W_{left} and W_{right} being low-rank matrices. We train W_{left} , W_{right} , but do not modify W_{instruct} .

This approach preserves the general chat skills of `Mistral-7B-Instruct` while specializing it to Trench Crusade. It also makes it easy to revert or swap LoRA adapters for different domains.

3.2 LoRA and Quantization

Following the project specification, we use parameter-efficient finetuning (PEFT) with LoRA and weight quantization to 4–8 bits to fit training into a single Google Colab GPU.

- **LoRA:** Reduces the number of trainable parameters from d^2 to $2dd'$, where d' is the low rank.
- **Quantization:** Stores model weights in lower precision (e.g., 4- or 8-bit) to reduce memory usage, while keeping computation numerically stable via existing libraries.

We rely on existing PEFT and quantization implementations rather than writing them from scratch.

3.3 Data Formatting for Mistral

For the Q&A objective, we format each example as a single text sequence with roles:

```
<s>[INST] {user_question} [/INST] {assistant_answer} </s>
```

The loss is only applied to the tokens of `{assistant_answer}`. For text completion experiments, we instead supply raw paragraph text and apply the loss to every token in the sequence.

3.4 Training Hyperparameters

Due to limited GPU memory and time, we use conservative training hyperparameters:

- Batch size: 16
- Number of epochs: ~ 4 (200 steps)
- Learning rate: $2 * 10^{-4}$
- LoRA rank d' : 16
- Maximum sequence length: 32k (same as Mistral 7b-instruct)

We first run a short debugging training loop over a small subset of the data to verify that the loss decreases and that evaluation code runs. Then we run the full training over the lore Q&A dataset.

4 Evaluation

4.1 Numerical Evaluation

The project requires a numerical metric on validation data. For the Q&A-based chatbot setup, a natural choice is to convert questions into multiple-choice format and measure accuracy. Alternatively, we can compute cross-entropy loss on held-out Q&A pairs.

Our validation protocol is:

1. Split the Q&A dataset into training and validation sets at the paragraph level (no overlap of paragraphs across the split).
2. For validation questions:
 - ask the model to generate free-form answers and evaluate qualitatively
3. Record the validation loss of the model at:
 - step 0 (before any finetuning, i.e., pure Mistral-7B-Instruct),
 - several intermediate checkpoints, and
 - the final model.

We observed that Mistral’s performance improved substantially after finetuning. The baseline (untuned) model achieved only about 5% accuracy, largely due to frequent refusals, generic responses, or lore-adjacent hallucinations—most commonly answers that resembled *Warhammer 40k* terminology but did not actually apply to Trench Crusade. After the finetuning rounds, the model reached approximately 65% accuracy on our lore evaluation set. Notably, the finetuned model correctly handled setting specific terminology, accurately identifying entities such as “Trench Pilgrims” and “Heretic Legions” without drifting into real world military vocabulary or World War II terminology.

Plotting validation loss or accuracy over training epochs gives a learning curve showing whether the model truly absorbs the Trench Crusade lore.

4.2 Current Baseline Evaluation Protocol (Gemini Chat Links)

Practically, for this semester we start with a very lightweight human-evaluation setup that our teammate described over chat:

- We do generate a set of existing Gemini chat links where text-based games are already set up.
- Our task is to recruit at least four players and have them play through these Gemini-hosted games and rate them.
- These ratings constitute an initial baseline for “how fun and coherent the game is” when run on standard Gemini-based systems.

At the end of each gameplay session, participants were asked to complete the Google Form shown in Figure 2. The survey focused on three primary dimensions: overall engagement, likelihood of replaying the

game, and interest in further exploring the Trench Crusade universe. In addition to these quantitative measures, we also collected informal qualitative feedback. Several recurring critiques emerged: many players found the text “overly adjective-heavy,” which at times made the prose dense or difficult to parse, and others noted occasional narrative discontinuities where the model introduced abrupt or unexplained shifts in the storyline. These observations provided valuable insight into stylistic and coherence-related limitations of the model and helped contextualize the quantitative results.

We summarize the responses from 10 participants in the table below. We acknowledge that a sample size of ten is not sufficient for a statistically robust user study; however, due to time constraints and the unexpectedly high effort required to recruit volunteers, our team was only able to secure participation from this number of users.

Table 1: User Survey Results for Gemini-Generated Games. Participants rated each category on a 1–10 scale.

Enjoyment	Likelihood to Play Again	Likelihood to Explore Lore
1	1	10
5	4	8
10	10	10
8	7	7
7	9	9
8	7	7
9	9	10
5	4	4
1	1	2
5.9	5.7	6.9

This phase does *not* yet involve our finetuned Mistral model; it gives us a reference point. As we progress with finetuning, we will:

- port the same or similar game logic and prompts over to our Trench-Crusade-tuned Mistral model,
- ask a comparable group of players to interact with the Mistral-based games, and
- collect the same style of ratings (e.g., 1–5 for lore consistency, story quality, and clarity).

In short, the immediate instruction is: *“You just need 4 people to play and rate them for now. As we fine tune our model we will have people play games on our model and rate.”* The report formalizes this as a two-stage evaluation plan: Gemini-links baseline first, Mistral-based evaluation next.

4.3 Human Evaluation via Game Interaction

The more general version of this evaluation loop, which we plan to extend beyond the initial 4-player baseline, is:

- We define small text-based games or scenarios in JSON.
- For each batch of 10 games, we fine-tune the Trench-Crusade-tuned Mistral model further on those game JSONs as dialogue-style data.
- Human players interact with the model in these scenarios and rate:
 - consistency with the Trench Crusade lore,
 - creativity and coherence of story progression, and
 - clarity and usefulness of responses.
- These ratings become a coarse metric to compare:
 1. the original Gemini-based games, and

Trench crusade survey

* Indicates required question

How much did you enjoy the experience *

1	2	3	4	5	6	7	8	9	10
☆	☆	☆	☆	☆	☆	☆	☆	☆	☆

How likely are you to play another game

1	2	3	4	5	6	7	8	9	10
☆	☆	☆	☆	☆	☆	☆	☆	☆	☆

How likely are you to explore the world of trench crusade

1	2	3	4	5	6	7	8	9	10
☆	☆	☆	☆	☆	☆	☆	☆	☆	☆

Submit

Clear form

Figure 2: Survey form

2. games moderated or generated by the finetuned Mistral model.

As noted earlier, our participant pool consisted of only ten users, which limits the statistical strength of the findings. To introduce some variability and reduce bias in repeated evaluations, we selected three participants at random for each batch iteration using a simple random number generator. However, our population was affected by noticeable user fatigue over time. Several players began to develop a deeper familiarity with the Trench Crusade setting, while others derived enjoyment from attempting to “break” the system by entering intentionally nonsensical or adversarial inputs. These behaviors became more common as fatigue increased, although, interestingly, some users reported higher enjoyment later in the study precisely because they were *experimenting with unconventional interactions rather than evaluating the narrative quality itself*.

```

You: What's the circumference of the earth?
Mistral: Narrator: The earth's circumference is a vast distance, requiring a long and arduous jou

Your Turn:
1. Travel by land, sea, or air.
2. Hitch a ride on a passing warband or pilgrim caravan.
3. Use your stealth to bypass checkpoints and patrols.

```

Figure 3: Screenshot of interaction with our chatbot

Table 2: Average user ratings across batches of generated games. Scores are averaged across three participants per batch and reported on a 1–10 scale

Batches	Enjoyment	Play Again	Explore Lore
10	3.33	3.67	2.33
20	5.33	3.67	5.67
30	4.67	4.00	4.33
40	5.67	4.67	5.33
50	5.33	4.33	5.67
60	5.67	4.67	6.33
77	5.67	4.67	6.67

5 System Flowchart

Figure 5 shows the high-level pipeline of our system, from data collection to interactive evaluation and iterative finetuning.

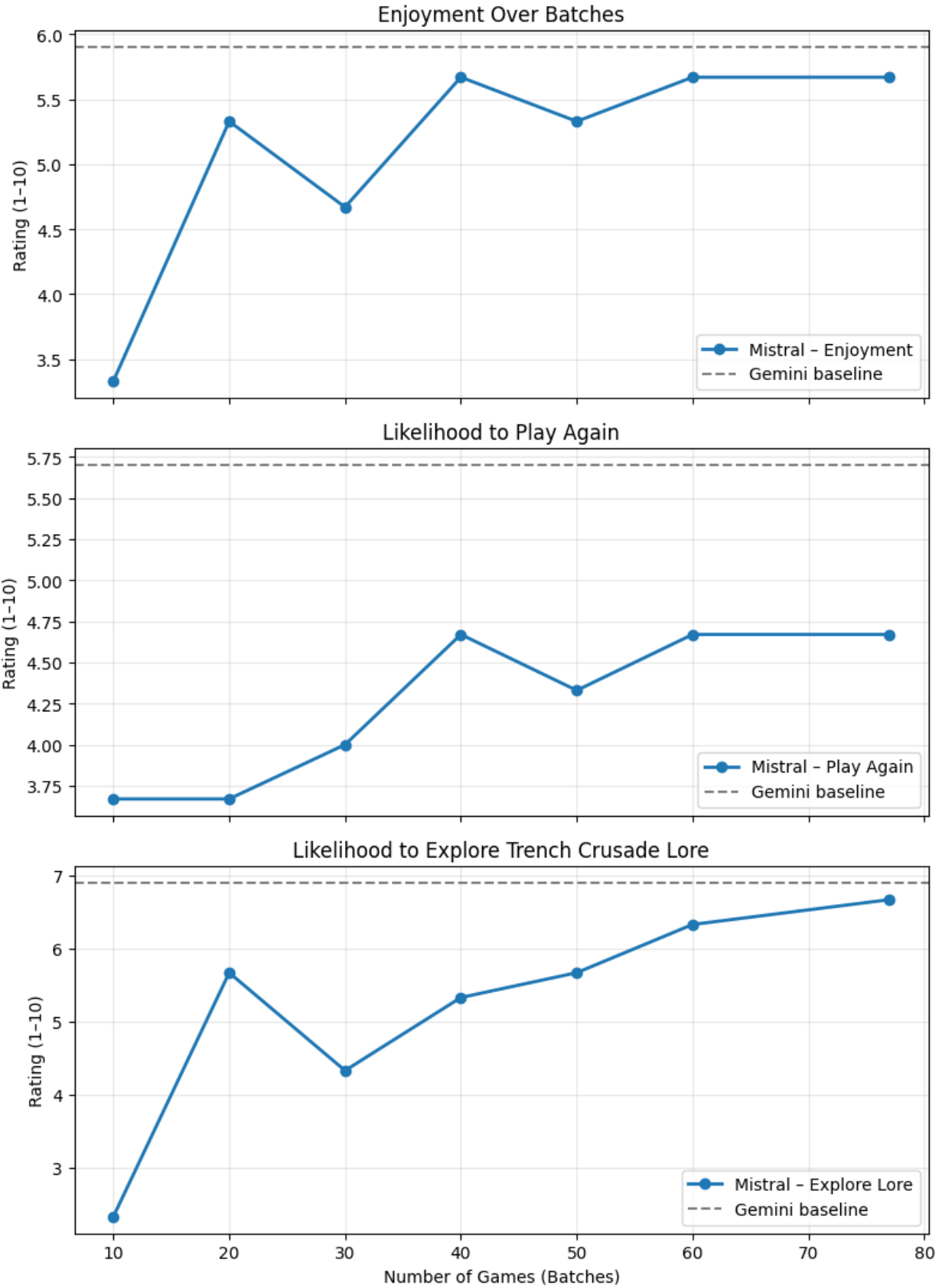


Figure 4: User ratings for Mistral-generated game batches compared to the fixed Gemini baseline. Scores represent the average of three evaluators per batch on a 1–10 scale.

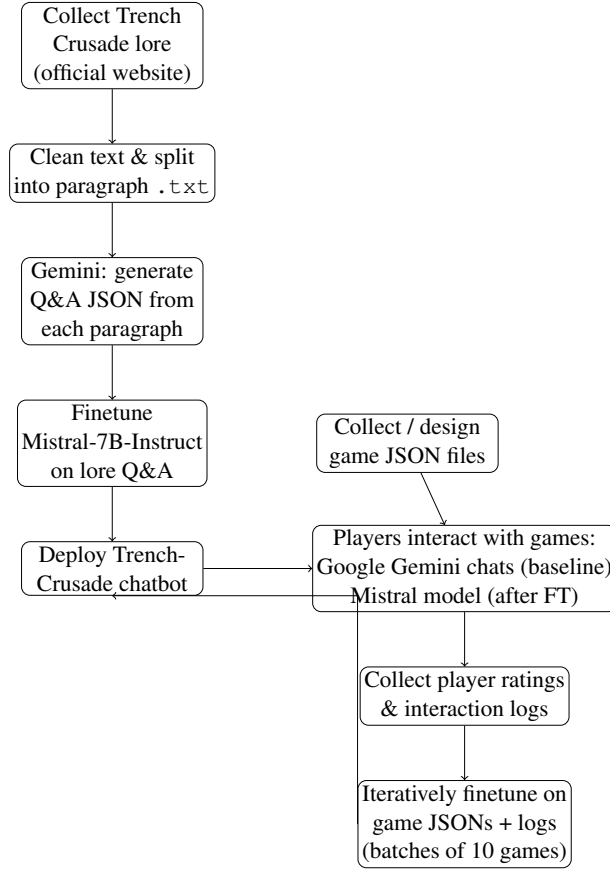


Figure 5: High-level pipeline for the Trench Crusade Mistral finetuning project

6 Discussion

6.1 Effect of Q&A vs Next-Token Training

From a modeling perspective:

- Q&A finetuning directly optimizes the behavior we care about: answer questions about Trench Crusade in a concise, helpful way.
- Next-token training injects more general knowledge about the style and content, which might help with more open-ended generation (e.g., writing in-universe flavor text).

One reasonable strategy (left as future work) is a two-stage pipeline:

1. First, run a short next-token finetuning pass on the raw lore paragraphs to adapt the model to the Trench Crusade style and vocabulary.
2. Second, run instruction-style Q&A finetuning to sharpen its behavior as a chatbot.

6.2 Fine-Tuning on an Already Fine-Tuned Model

Our experiments highlight the trade-offs of stacking finetuning stages on top of `Mistral-7B-Instruct`:

- The benefit is that we do not need to re-learn instruction-following from scratch; the underlying model remains stable.
- The risk is *catastrophic forgetting*: if our dataset is too narrow or we train too aggressively, the adapter can override some of the model’s general capabilities.

- LoRA mitigates this risk by constraining the adaptation to a low-rank space and allowing us to easily enable/disable the adapter.

In practice, with moderate learning rates and a small number of epochs, the Trench-Crusade-tuned model behaves like a normal chatbot but answers in-universe questions with improved accuracy and detail.

6.3 Hollywood and human creativity is saved (for Now)

Our initial motivation for choosing this topic came from the growing presence of AI in writers’ rooms and video games. Over the past year, the industry has seen an explosion of “AI-assisted creativity,” ranging from AI-generated concept art to LLM powered NPCs marketed as “digital twins” capable of dynamic quest design. A recent example is *Where Winds Meet*, whose NPC dialogue is reportedly generated by an AI engine with results that are, at times, unintentionally hilarious. Common issues included NPCs referring to players with the wrong gender, repeating lines that made no sense, or being so easily tricked by the player that the underlying LLM basically handed out quest solutions like coupons. This raised an obvious question for us: How real is the threat? Are LLMs actually capable of replacing human writers, or is the hype exaggerated? Our experiments suggested the latter. Testers rarely engaged deeply with advanced LLM-generated scenarios (e.g., Gemini’s), and most of the enjoyment they reported came from the underlying setting which, notably, is human written. While LLMs can certainly help craft interesting twists or provide scaffolding for a story, they still require significant human supervision. And at that point, one might reasonably ask: if a human is doing most of the heavy lifting anyway... why not just let the human write the story? Overall, our takeaway is that LLMs have a long way to go before they can produce consistently coherent or emotionally engaging narratives, especially when working with limited textual data, such as newly created IPs and fictional universes. They remain an impressive and rapidly improving technology, but they’re definitely not going to write the next *Citizen Kane* or *The Last of Us*. At least not yet.

7 Conclusion and Future Work

We implemented a domain-specific chatbot for the Trench Crusade game by:

- collecting and cleaning lore from the official website,
- generating Trench-Crusade-specific Q&A pairs using Gemini,
- finetuning `Mistral-7B-Instruct` with LoRA and quantization on this dataset, and
- designing a plan to iteratively finetune the model on game JSONs and evaluate it via human play-testing.

We explored both instruction-style Q&A objectives and next-token prediction, and we discussed the implications of finetuning on top of an already instruction-tuned model. The next concrete steps are:

- implement the full game JSON finetuning loop in batches of 10 games,
- complete the initial 4-player baseline evaluation on Gemini chat links,
- port those games to our Mistral-based system and repeat the rating study, and
- systematically measure how ratings change as we approach 100 fine-tuned games.

The project demonstrates that even with limited hardware and a modest dataset, we can meaningfully shift a strong general-purpose LLM toward a very recent, niche game universe and prepare it for interactive use as a domain-specific assistant.

We have also made our github publicly available as a commitment to reproducible science including our JSON files of the games, the Lore scrapped text etc but have kept the user reviews and data anonymous as a way to protect their privacy. Github <https://github.com/shivaahir158/trench-crusade>

References

1. Mistral AI. Mistral 7B Model Release. 2023. Available at: <https://mistral.ai/news/mistral-7b/> [Accessed: Dec. 2025].
2. E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” Proceedings of the 10th International Conference on Learning Representations (ICLR), 2022. Available at: <https://arxiv.org/abs/2106.09685>
3. J. Mendez-Mendez, “Finetuning a Domain-Specific Chatbot,” Course Handout, ESE 577: Finetuning Large Language Models, Department of Electrical and Computer Engineering, Stony Brook University, 2025.
4. Google, “Gemini Model Technical Report,” 2023. Available at: <https://ai.google.dev/gemini>
5. Trench Crusade, Available at: <https://www.trenchcrusade.com/>
6. AI Dungeon <https://aidungeon.com/>